



UNIVERSIDAD AUTÓNOMA DE QUERÉTARO
FACULTAD DE INFORMÁTICA

INTELIGENCIA ARTIFICIAL

-PROYECTO FINAL-

ALGORITMO DE CLASIFICACIÓN DE INSECTOS MEDIANTE PROCESAMIENTO DIGITAL DE IMÁGENES.

INTEGRANTES:

LUIS DAVID GLORIA ALVAREZ - 282653

MARIA FERNANDA BARRON GUZMAN - 325805

DANIEL ISAIAS GONZALEZ RAMIREZ - 325862

ERICK MANUEL HERNANDEZ SANTOS - 253226

DAVID CRUZ MELCHOR – 325768

-PROYECTO-

En el ámbito de la biología la identificación y clasificación de especies de insectos juega un papel crítico para el monitoreo de ecosistemas y el estudio de la biodiversidad. Normalmente, esta clasificación se ha realizado mediante una inspección visual humana, sin embargo, esta clasificación humana es propensa a muchos errores, pérdida de información y enfrenta límites de escalabilidad cuando se presentan miles de muestras.

Por lo que mediante el siguiente proyecto se implementará un algoritmo de procesamiento que simula escenarios de baja resolución mediante submuestreo digital. Para contrarrestar la pérdida de información y reconstruir las estructuras biológicas esenciales, se desarrollará un algoritmo de interpolación. Esta técnica matemática estimara la intensidad de los nuevos píxeles a partir de una matriz de 4x4 píxeles originales, así suavizando los bordes y manteniendo la continuidad de las líneas antes del análisis.

Una vez que se optimice la calidad de la imagen, el sistema reducirá la dimensionalidad de los datos visuales para extraer las propiedades de la muestra. Mediante el procesamiento en escala de grises, se calcularán cuatro características: la media del brillo, la desviación (contraste), la cantidad de bordes visualizados y la entropía o complejidad de la textura de las primeras muestras. Este enfoque permitirá mapear estructuras complejas, como las venas en las alas de las abejas, las moscas y las libélulas. También servirá para entrenar el algoritmo y prepararlo para la nueva muestra.

Para llevar a cabo la agrupación de las muestras el algoritmo utilizara un parámetro llamado “confianza”, el cual funcionara como un límite, si las cuatro características de una muestra son lo suficientemente parecidas a las de un grupo y no rebasan el limite de confianza, el grupo aceptara la muestra.

Finalmente, el algoritmo ajustara su tasa de aprendizaje, actualizando el promedio del grupo y aprender de la nueva muestra que se ingresara. Ya que el algoritmo está entrenado y tiene sus grupos definidos, será capaz de evaluarla y agruparla correctamente y si la muestra es muy distinta, optara por crear un nuevo grupo.

DESCRIPCION

Una vez hecho el planteamiento, se procedió a llevar a cabo el proyecto, el cual se describe a continuación:

- **MAIN**

```
1 % Limpia pantalla, borra variables, imagenes, graficas, variables auxiliares
2 clc
3 close all
4 clear all
5
6 fprintf('EMPEZAMOS\n');
7 disp('=====');
8
9 % Carga libreria para poder trabajar con procesamiento de imagenes (imread, imshow... etc)
10 pkg load image
11
12 % Obtenemos nombre de las imagenes
13 archivos = dir('MUESTRAS/*.jpg');
14
15 baseDatos = []; % Se guardan las caracteristicas numericas de las imagenes
16 nombresImagenes = {}; % Se guardan los nombres de las imagenes
17
18 % Recorremos todas las imagenes
19 for i=1:length/archivos
20     nombre = archivos(i).name; % Obtenemos el nombre
21
22     ruta = strcat('MUESTRAS/', nombre); % Obtenemos la ruta
23
24     imagen = imread(ruta); % Cargamos la imagen como matriz de valores
25
26     imagen = imresize(imagen,0.1); % Reducimos la calidad de la imagen
27     % para realizarla después con interpolación
28
29     imagenInterpolada = imresize(imagen, 2, 'bicubic');
```

Se empezó por ejecutar una limpieza (borrando la consola, variables en memoria y ventanas gráficas abiertas) para asegurar que el programa corra desde cero sin interferencias de ejecuciones anteriores.

Tras cargar la librería necesaria para manipular imágenes, el sistema escanea automáticamente la carpeta MUESTRAS/ para enlistar todas las fotografías .jpg que se van a analizar e inicia arreglos vacíos (baseDatos y nombresImagenes) donde posteriormente se guardará la información matemática extraída de cada muestra.

Después arranca un ciclo que procesará cada imagen de la lista una por una. Primero, el algoritmo simula intencionalmente un escenario adverso reduciendo drásticamente la calidad de la imagen de la muestra al 10% de su tamaño original mediante la función imresize. Inmediatamente después, intenta reconstruir la geometría de las alas o patas perdidas aplicando una interpolación bicúbica (usando 'bicubic') para duplicar el tamaño de la imagen de la muestra, dejándola lista para la extracción de sus cuatro características.

```

main.m x
36 % Extraer características de la imagen
37 % Extraer características de la imagen
38
39
40
41
42
43 fprintf('Características imagen: ');
44 disp(i);
45 disp(caracteristicas);
46
47 % Guardar características de las imágenes (las apilamos en filas)
48 baseDatos = [baseDatos; caracteristicas];
49
50 % Guardar nombre imagen
51 nombresImágenes(i) = nombre;
52
53 endfor
54
55 disp('=====');
56 disp('ENTRENANDO CLASIFICADOR EM');
57 disp('=====');
58 % Entrenar modelo de clasificación
59 % Entrenar modelo de clasificación
60
61 [mu, varianza] = entrenarEM(baseDatos);
62 % Los datos los guardamos en una matriz
63 % Los datos los guardamos en una matriz
64 % Los datos los guardamos en una matriz
65

```

La imagen recién reconstruida (imagenInterpolada) se envía a la función extraerCaracteristicas. La imagen deja de ser interpretada como una cuadrícula visual y se convierte en un vector de cuatro valores (brillo, contraste, bordes y entropía).

La instrucción baseDatos = [baseDatos; caracteristicas]; funciona como una tabla que crece hacia abajo, apilando los datos de cada nueva imagen como una fila adicional. Guarda el nombre del archivo original (nombresImágenes) para saber exactamente a qué insecto le pertenece cada fila de números.

Una vez que el ciclo termina, se comienza con la ejecución del modelo de clasificación: [mu, varianza] = entrenarEM(baseDatos). El sistema le entrega la tabla de datos completa al algoritmo de aprendizaje. Después de analizar las similitudes, el algoritmo devuelve sus conclusiones en dos variables: mu (una matriz con los promedios de cada grupo biológico que logró formar) y varianza (los márgenes de tolerancia de cada grupo).

```

main.m x
66 % for i=1:size(mu,1) % El ciclo va a ser de la cantidad del número de filas de mu (grupos)
67
68 fprintf('\nGRUPO %d\n', i);
69
70 fprintf('Media Intensidad : %.2f\n', mu(i,1)); % (BRIJO, BRILLO)
71 fprintf('Desviacion : %.2f\n', mu(i,2));
72 fprintf('Cantidad Bordes : %.2f\n', mu(i,3));
73 fprintf('Entropia : %.2f\n', mu(i,4));
74
75 endfor
76
77 disp('=====');
78 disp('CLASIFICACION DE IMAGENES');
79 disp('=====');
80
81 % for i=1:size(baseDatos,1) % Cantidad de filas de la base de datos (todas las imágenes)
82
83 % Extraer características de la imagen
84
85 grupo = clasificarImagen(baseDatos(i,:), mu, varianza);
86 % Se devuelve la fila i, todos los valores, todos los grupos del mu, que son los datos
87 % de cada grupo, del cual vamos a clasificar la imagen
88
89 fprintf('%s ---> GRUPO %d\n', nombresImágenes(i), grupo);
90
91 endfor
92
93 disp('=====');
94 disp('PRUEBA CON NUEVA IMAGEN');
95 disp('=====');
96
97 % Ruta a la nueva imagen
98 rutaNueva = 'abejaMiel4.jpg';

```

Una vez que el algoritmo de agrupamiento finalizó su trabajo, este ciclo (for i=1:size(mu,1)) se encarga de imprimir en consola los datos de cada grupo. Por cada grupo creado, el

sistema imprime los valores promedio almacenados en la matriz mu, mostrando cuál es su intensidad esperada, su desviación, la cantidad de bordes y su entropía promedio.

El sistema recorre nuevamente todas las filas de la base de datos original (baseDatos) y las envía una por una a la función clasificarImagen. Le pasa las características de la imagen (baseDatos(i,:)), junto con mu y la varianza o flexibilidad de los grupos que acaba de entrenar. Esto para imprimir el nombre de la imagen frente al número de grupo al que el sistema decidió que pertenece.

```
main.m
100 if(exist(rutaNueva, 'file'))
101
102 nueva = imread(rutaNueva);
103
104 nueva = imresize(nueva,0.1);
105
106 nuevaInterpolada = imresize(nueva,2,'bicubic');
107
108 caracteristicasNueva = extraerCaracteristicas(nuevaInterpolada);
109 fprintf('Caracteristicas imagen: \n');
110 disp(caracteristicasNueva);
111
112 grupoNuevo = clasificarImagen(caracteristicasNueva, mu, varianza);
113
114 fprintf('\nResultado:\n');
115
116 if(grupoNuevo == -1)
117
118     disp('La imagen NO pertenece a ningun grupo conocido');
119
120 else
121
122     fprintf('La imagen pertenece al GRUPO %d\n', grupoNuevo);
123     fprintf('\nGRUPO %d\n', grupoNuevo);
124
125     fprintf('Media Intensidad : %.2f\n', mu(grupoNuevo,1));
126     fprintf('Desviacion : %.2f\n', mu(grupoNuevo,2));
127     fprintf('Cantidad Bordes : %.2f\n', mu(grupoNuevo,3));
128     fprintf('Entropia : %.2f\n', mu(grupoNuevo,4));
129
130 endif
131
132 else
133
134     disp('No existe prueba.jpg');
135
136 endif
```

En esta última parte del main, primero se inicia un candado if(exist(rutaNueva, 'file')) para asegurarse que la imagen de prueba exista en el directorio. Si la encuentra la lee y le hace las mismas pruebas que las imágenes de entrenamiento. A la imagen recién reconstruida se le aplica la función extraerCaracteristicas para obtener su vector de 4 dimensiones (brillo, contraste, bordes y entropía). El script imprime estos valores en pantalla.

Por último, el sistema ejecuta la función clasificarImagen, enviándole los datos de la nueva imagen y todo el conocimiento que adquirió previamente (mu y varianza). Si la función devuelve un -1, significa que las características de la imagen nueva superaron el umbral de confianza de los grupos existentes e imprime: “La imagen NO pertenece a ningún grupo conocido”. Pero si la función encuentra un grupo compatible, el código entra al bloque else. Imprime a qué grupo pertenece la imagen y vuelve a imprimir los promedios (Media Intensidad, Desviacion, Cantidad Bordes, Entropia) de ese grupo.

- **extraerCaracteristicas:**

```

1 function caracteristicas = extraerCaracteristicas(imagen)
2
3 % Carga libreria para poder trabajar con procesamiento de imagenes (images, image, etc)
4 pkg load image
5
6 % Si la imagen esta a color (3 canales) la convertimos a escala de grises, esto
7 % simplifica el analisis
8 if(size(imagen,3)==3)
9     imagen = rgb2gray(imagen);
10 endif
11
12 % Convertir a double para poder trabajar mejor con sus valores
13 imagen = double(imagen);
14
15 % CARACTERISTICA 1: Calcula el promedio de todos los elementos de la matriz
16 % para poder definir que tan claro o oscuro es. Alto valor = claro, bajo = oscuro
17 mediaIntensidad = mean(imagen(:));
18
19 % CARACTERISTICA 2: Esto define que tanto varían los pixeles (desviación estándar).
20 % para saber que tanto contraste tiene una imagen
21 desviacion = std(imagen(:));
22
23 % CARACTERISTICA 3: Esto detecta bordes, al final obtenemos la cantidad total de bordes
24 % que hay para poder distinguir los objetos entre ellos con muchas líneas o pocas líneas
25 bordes = edge(uint8(imagen), 'canny');
26 cantidadBordes = sum(bordes(:));
27
28 % La entropia mide que tan desordenada o compleja es una imagen, cantidad de info visual
29 % (muchos colores, formas, texturas)
30 entropiaImagen = entropy(uint8(imagen));
31
32 % Devolver final con las 4 caracteristicas
33 caracteristicas = [mediaIntensidad desviacion cantidadBordes entropiaImagen];
34
35 endfunction

```

Primero el algoritmo revisa si la imagen esta a color, con la línea `if(size(imagen,3)==3)`, de ser así, la convierte a escala de grises usando `rgb2gray`, esto porque analizar una sola capa de grises es computacionalmente mucho más rápido y efectivo para detectar formas que analizar tres capas de colores.

Enseguida se extraen las cuatro características que definen como se ven las imágenes:

1. Se calcula el promedio de todos los píxeles de la imagen con la función “mean”.
2. Se utiliza la función “std” para medir qué tanto varían los colores de los píxeles respecto a la media.
3. Se utiliza “Canny” (`edge(..., ‘canny’)`), para detectar los bordes y con la función `sum`, cuenta cuántos píxeles de borde se detectaron en total.
4. Se usa la función “entropy” para medir el nivel de "desorden" o cantidad de información en la textura de la imagen.

Finalmente, una vez calculadas estas cuatro características, se agrupan todos dentro del arreglo “caracteristicas”, para después devolverlos al main.

- **entrenarEM:**

```

1 function [mu, varianza] = entrenarEM(datos)
2 rho = 0.95; %El rho nos ayuda a decidir que tanto valor va a tener el resultado
3 % pasado y el nuevo resultado. Con rho 0.9 le da el 90% del valor al pasado y 1 al promedio
4
5 confianza = 200; %Que tan flexible es el grupo. Si la distancia es menor a 200
6 % se parte del grupo. Si no, se crea un nuevo grupo. Podemos ajustar la confianza para
7 % modificar la cantidad de grupos que se van a crear, mientras mas confianza menos grupos
8
9 % Creamos los grupos de mu y varianza
10 mu = [];
11 varianza = [];
12
13 for i=1:size(datos,1) % Recorre la cantidad de filas (cantidad de imagenes)
14
15     muestra = datos(i,:); % Tomamos los datos de todas las columnas de una fila
16
17     D = zeros(size(mu,1),1); %Creamos una matriz de (filas en mu) * 1 para guardar la
18     % distancia que cada imagen tiene con cada grupo
19
20     for j=1:size(mu,1) % Recorre todos los grupos que se han generado
21
22         distancia = 0;
23
24         for k=1:size(datos,2) %Recorre la cantidad de columnas de cada imagen (caracteristicas)
25             distancia += ((muestra(k)-mu(j,k))^2) / varianza(j,k);
26             % Tomamos la muestra y la restamos al promedio que hay en mu y elevamos la cantidad
27             % al cuadrado para evitar negativos y hacemos divisiones grandes y dividimos entre la varianza
28
29         endfor
30
31         D(j) = distancia; %Guardamos distancia
32
33     endfor
34
35 endfunction

```

Al inicio se establecen “rho” y “confianza”, que son las dos variables que van a determinar el comportamiento de la red. Con “rho” se define la tasa de aprendizaje y nos indicara que tanto porcentaje le damos al conocimiento previo y al nuevo. En “confianza” se define la tolerancia geométrica entre las imágenes, si la distancia supera 200, se agrupará la imagen aparte.

Después el algoritmo inicia un ciclo para evaluar cada imagen, aislando sus cuatro características principales para compararlas matemáticamente con todos los grupos ya creados. Al terminar de evaluar las 4 características, guarda el puntaje total de distancia en la posición correspondiente del marcador (D(j) = distancia;).

```

31     D(j) = distancia; %Guardamos distancia
32
33 endfor
34
35 Dc = D <= confianza; % Creamos un vector de booleanos para ver que grupos cumplen
36
37 if(sum(Dc)==0) %Si ningún grupo cumple la imagen
38
39     mu = [mu; muestra]; % Se crea un nuevo grupo
40     varianza = [varianza; ones(1,size(datos,2))*10];
41     % Creamos el vector de la varianza de los grupos
42
43 else %Si pertenecen a un grupo
44
45     for j=1:size(Dc,1) %Se recorren todos los grupos
46
47         if(Dc(j)==1) %Si acepta la imagen
48
49             for k=1:size(datos,2) %Se recorren las características
50
51                 mu(j,k) = rho*mu(j,k) + (1-rho)*muestra(k); %Actualizamos el promedio
52
53                 varianza(j,k) = rho*varianza(j,k) + (1-rho)*(mu(j,k)-muestra(k))^2;
54                 % Se suma el cuadrado de la distancia de cada imagen a la varianza y se divide entre el número de imágenes
55
56             endfor
57
58         endfor
59
60     endfor
61
62 endif
63
64 endfor
65
66 endfunction

```

Por ultimo se genera un vector booleano (Dc) que verifica de manera instantánea si los puntajes de distancia calculados en el paso anterior lograron quedar por debajo del límite de confianza. Si la suma de ese vector es cero (es decir, ningún grupo aceptó la imagen porque es muy distinta a lo conocido), el algoritmo crea un grupo nuevo. Registra las características de esta imagen aislada como el nuevo promedio (mu) y le asigna una varianza predeterminada.

Si la imagen fuera aceptada por un grupo ya existente, el algoritmo entra al bloque “else” para aprender de ella y utilizando la tasa de aprendizaje (rho), recalcula la media y su varianza.

- **clasificarImagen:**

```
1 function grupo = clasificarImagen(caracteristicas, mu, varianza)
2     % Clasifica las características de la imagen y de cada grupo con sus medias y varianzas
3
4     confianza = 200; %Nivel de confianza para aceptar la imagen, si la rechazamos, no aprendemos
5
6     D = zeros(size(mu,1),1); %Se crea un vector para guardar distancias con cada grupo
7
8     for j=1:size(mu,1) % Recorremos cada grupo
9
10        distancia = 0;
11
12        for k=1:size(caracteristicas,2) %Recorremos cada característica
13
14            distancia += ((caracteristicas(k)-mu(j,k))^2) / varianza(j,k);
15            %Calculamos la distancia con el grupo
16
17        endfor
18
19        D(j) = distancia; %Guardamos la distancia total con el grupo
20
21    endfor
22
23    [valorMinimo, indice] = min(D); %Buscamos el menor valor entre los valores guardados
24    % y el índice que les corresponde
25
26    if(valorMinimo <= confianza) % Si el grupo al que más se acerca no sobrepasa el
27        % nivel de confianza, aceptamos la imagen dentro del grupo
28        grupo = indice;
29    else
30        grupo = -1; %Si todos los grupos sobrepasaron el nivel de confianza se crea dentro de ninguno
31    endif
32
33 endfunction
```

El objetivo principal de “clasificarImagen” es tomar los datos de la nueva muestra y decidir a qué grupo pertenecen.

El algoritmo recibe las cuatro características de la nueva imagen y utiliza la misma fórmula matemática del entrenamiento para medir qué tan lejos está respecto a mu y varianza de todos los grupos existentes. Una vez que evalúa los grupos encuentra la distancia mas pequeña y el índice que le corresponde. Por último, evalúa el nivel de “confianza”, si cumple la condición, el sistema acepta la imagen y devuelve el número del grupo. Si no la cumple, asume que es una muestra desconocida y la rechaza devolviendo un -1.

INTERPOLACION BICUBICA DESDE CERO

Para explicar mejor el proceso que ocurre al interpolar una imagen se creo una alternativa, en donde, en lugar de usar la función “imresize” la cual hace que se interpole en milésimas de segundo, se crearon “bicubicaEscala” y “bicubica”. Con estos algoritmos se realiza la interpolación manualmente, aunque el inconveniente es que tardara mucho mas tiempo en clasificar las muestras. También se creó una prueba llamada “interpolacionBicubica”. A continuación, se describen estas tres funciones:

- **bicubicaEscala:**

```
1 function imgF = bicubicaEscala(img, escala)
2 % Se calcula el tamaño de la matriz de los pixeles de la imagen
3 [h, w] = size(img);
4 % Se calcula el tamaño de la matriz de los pixeles de la imagen a la escala que se quiere lograr
5 nh = round(h*escala);
6 nw = round(w*escala);
7 disp(nh);
8 disp(nw);
9 % Se crea una matriz para los pixeles de la imagen escalada
10 imgF = zeros(nh,nw);
11 % Se calcula la relación entre la imagen original y la imagen que se va a escalar
12 % Se calcula la relación de 1 pixel de la original a cuantos pixeles equivale en la escalada
13 re = (h-1)/(nh-1);
14 ce = (w-1)/(nw-1);
15 if(re ~= 0.5 && ce ~= 0.5)
16     re = 0.5;
17     ce = 0.5;
18 endif
19 % Se recorre cada pixel con valores en base de la matriz de la imagen resultante(nueva)
20 for i=1:nh
21     for j=1:nw
22         % Se calcula la relación que nos ayudará a interpolamos el nuevo valor
23         x = (i-1)*re+1;
24         y = (j-1)*ce+1;
25         x1 = floor(x);
26         x2 = max(x1-1,1);
27         x3 = min(x1+1,h);
28         x4 = min(x3+1,h);
29         y1 = floor(y);
30         y2 = max(y1-1,1);
31         y3 = min(y1+1,w);
32         y4 = min(y3+1,w);
33         disp(y1);
34         disp(y2);
```

Primero, el algoritmo toma las medidas de alto y ancho de la matriz original ([h, w] = size(img);). Se calcula las dimensiones exactas que deberá tener la nueva matriz multiplicando esos valores por la variable “escala”. Con imgF = zeros(nh,nw);, crea espacios en blanco que se irán rellendo con nuevos pixeles.

Para poder llenar los espacios en blanco, el programa necesita saber a qué punto de la muestra original equivale cada nuevo espacio. Las variables “re” y “ce” calculan esta proporción entre pixeles. Con los ciclos “for” se calculan las coordenadas de “x”, “y”, de cada espacio en blanco y como no puede haber medios pixeles, en caso de caer en uno, se redondea usando “floor” y formando una cuadrícula de 4x4 pixeles alrededor de ese punto (x1, x2, x3, x4, y1, y2, y3, y4).

Para saber el color de esos pixeles rellenos, se va a la imagen original con Q11 = img(x1,y1); y se extrae la intensidad (brillo/color) de cada uno de esos 16 puntos. Q representa el "peso" o valor visual del píxel. Por ultimo se manda a llamar a la función “bicubica”.

- **bicubica:**

```

6  %Funcion que calcula el valor de la funcion bicubica de acuerdo a la formula estipulada
7  disp('Calculando el valor de la funcion bicubica de acuerdo a la formula estipulada');
8  ni = length(xs);
9  nj = length(ys);
10
11  resultado_j = 0;
12  resultado_i = 0;
13
14  for i=1:ni
15      disp('Validando que se ejecuten todas las veces');
16      disp(i);
17      resultado_j = 0;
18      for j=1:nj
19          disp('Ejecuciones en j');
20          disp(j);
21          %Calculando el valor de la funcion bicubica de acuerdo a la formula estipulada
22          x_ = xs(i);
23          disp('X_');
24          disp(x_);
25          y_ = ys(j);
26          disp('Y_');
27          disp(y_);
28          q_ = qs(i,j);
29          disp('q_');
30          disp(q_);
31          %Calculando el valor de la funcion bicubica de acuerdo a la formula estipulada
32          lx_ = independiente(x, x_, xs);
33          disp('lx_');
34          disp(lx_);
35          ly_ = dependiente(y, y_, ys);
36          disp('ly_');
37          disp(ly_);
38          resultado_j += q_ * lx_ * ly_;
39          disp('resultado_j');
40          disp(resultado_j);
41      endfor
42      resultado_i += resultado_j;
43      disp('resultado_i');
44      disp(resultado_i);
45  endfor
46  disp('Probando el ultimo valor que obtiene resultado para 2');
47  disp(resultado_i);
48  Z = resultado_i;
49
50 endfunction

```

```

50 %Funcion que calcula el valor LX de acuerdo a la formula estipulada
51 function LX = independiente(x, x_, xs)
52     mult_xs = 1;
53     for i=1:length(xs)
54         if(x_ != xs(i))
55             mult_xs *= (x-xs(i))/(x_-xs(i));
56         endif
57     endfor
58     LX = mult_xs;
59 endfunction
60 %Funcion que calcula el valor LY de acuerdo a la formula estipulada
61 function LY = dependiente(y, y_, ys)
62     mult_ys = 1;
63     for i=1:length(ys)
64         if(y_ != ys(i))
65             mult_ys *= (y-ys(i))/(y_-ys(i));
66         endif
67     endfor
68     LY = mult_ys;
69 endfunction

```

Mediante la implementación de Polinomios de Interpolación de Lagrange, se crearon dos funciones llamadas “independiente” y “dependiente”, el objetivo de estas funciones es calcular qué tanto "peso" o influencia debe tener el color de un vecino específico sobre el nuevo punto blanco o vacío, dependiendo de la distancia a la que se encuentre. El código utiliza un ciclo “for” que multiplica iterativamente las distancias. La condición `if(x_ != xs(i))` es una regla matemática para evitar que la fórmula reste un punto consigo mismo y provoque un error de "división por cero".

En la función “bicubica” se reciben el punto imaginario donde está el hueco (x, y), las coordenadas de los 16 vecinos que lo rodean (xs, ys) y los 16 tonos de color de esos vecinos (qs). Se realizaron dos ciclos “for” anidados que iteran 4 veces cada uno (4x4 = 16 vueltas).

El algoritmo evalúa a cada uno de los 16 vecinos haciendo lo siguiente:

1. Toma la coordenada exacta del vecino (x_, y_) y el color que tiene (q_).
2. Manda a llamar a las dos funciones polinomiales de Lagrange para obtener el peso del vecino en el eje X (lx_) y en el eje Y (ly_).

3. Multiplica el color original del vecino por el resultado de ambas curvas de influencia ($q_ * lx_ * ly_$). Si el vecino está muy cerca del hueco, esta multiplicación conservará mucho de su color original; si está lejos, su aporte será mínimo.

Finalmente, el acumulador resultado_j += ... va sumando los 16 fragmentos de color ponderado. Al terminar los ciclos, ese sumatorio total se guarda en la variable de salida Z. Este valor es el color que el sistema utilizará para rellenar el píxel vacío en los espacios en blanco.

- **InterpolacionBicubica:**

```
24 %Cargamos una una de las canales de los rgb de la imagen
25 pkg load image
26
27 %Leemos la imagen correspondiente a una matriz de píxeles
28 imagen = imread("beteese.jpeg");
29 %Se muestra la imagen
30 imagenmc=imresize(imagen,0.05);
31 figure;
32 %Se muestra una ventana donde al manipularla podremos ver la imagen
33 imagesc(imagenmc);
34 %Se muestra de las píxeles originales de no ser así forzara en colores la imagen en el tamaño completo de la venta de la pantalla
35 daspect([1 1]);
36 %Se extrae el canal rojo
37 %Normalizamos con los valores de 0 a 255 a una escala del 0 a 1
38 r=double(imagenmc(:,:,1))/255;
39 %Se extrae el canal verde
40 g=double(imagenmc(:,:,2))/255;
41 %Se extrae el canal azul
42 b=double(imagenmc(:,:,3))/255;
43
44 %Interpolaciones para cada canal del color rgb
45 %Nos aseguramos que no se salga del rango de 0 a 1 que es el equivalente de 0 a 255
46 r_new=bicubicaEscala(r,2);
47 r_new=max(0,min(1,r_new));
48 g_new=bicubicaEscala(g,2);
49 g_new=max(0,min(1,g_new));
50 b_new=bicubicaEscala(b,2);
51 b_new=max(0,min(1,b_new));
52
53 %Volvemos a los valores originales de 0 a 255
54 %Para convertir de double a uint8 para que se pueda visualizar la imagen
55 r_new=uint8(round(r_new*255));
56 g_new=uint8(round(g_new*255));
57 b_new=uint8(round(b_new*255));
58
59 %Volvemos a agrupar cada canal de color del rgb
60 imagenes=cat(3, r_new, g_new, b_new);
61 figure;
62 %Mostramos la imagen escalada
63 imagesc(imagenes);
64 daspect([1 1]);
65
```

Este algoritmo es una prueba para demostrar que las funciones “bicubicaEscala” y “bicubica”. En este experimento se probó el escalado de grises, la reducción de tamaño a un 5% y también se probó el desarmar una imagen en sus tres canales RGB para llamar a “bicubicaEscala” tres veces distintas, una por cada color, para así adivinar los píxeles faltantes en las tres capas, revertir la normalización, multiplicar todo por 255 para regresar a la escala de colores estándar y concatenando las tres matrices en una sola muestra tridimensional, para lograr reconstruir la imagen a todo color.

RESULTADO

Una vez explicado a detalle el sistema, se procedió con la prueba final, la cual arrojo en consola lo siguiente:

```
EMPEZAMOS
=====
Caracteristicas imagen: 1
171.5734 21.3588 623.0000 5.6176
Caracteristicas imagen: 2
171.2715 21.4835 708.0000 5.5788
Caracteristicas imagen: 3
181.2393 21.1458 692.0000 5.5425
Caracteristicas imagen: 4
162.6292 20.8263 695.0000 5.6107
Caracteristicas imagen: 5
173.6405 24.6303 454.0000 5.5726
Caracteristicas imagen: 6
178.5968 25.3253 450.0000 5.5305
Caracteristicas imagen: 7
179.2148 25.0506 448.0000 5.4655
Caracteristicas imagen: 8
180.9946 24.7616 444.0000 5.4428
Caracteristicas imagen: 9
1.6907e+02 1.4308e+01 1.1580e+03 5.7701e+00
Caracteristicas imagen: 10
1.6991e+02 1.5247e+01 1.2650e+03 5.8630e+00
Caracteristicas imagen: 11
1.6901e+02 1.4076e+01 1.1080e+03 5.7390e+00
Caracteristicas imagen: 12
1.7318e+02 8.4280e+00 1.3430e+03 5.0104e+00
Caracteristicas imagen: 13
1.7456e+02 7.9113e+00 1.5130e+03 4.9147e+00
Caracteristicas imagen: 14
1.7342e+02 8.0714e+00 1.3920e+03 4.9049e+00
Caracteristicas imagen: 15
174.0765 12.4097 759.0000 5.0950
Caracteristicas imagen: 16
173.7749 12.8501 758.0000 5.1933
Caracteristicas imagen: 17
174.9810 15.4773 640.0000 5.5060
Caracteristicas imagen: 18
177.2130 18.3341 745.0000 5.7881
Caracteristicas imagen: 19
180.4773 15.3738 716.0000 5.5672
Caracteristicas imagen: 20
179.1875 17.3961 760.0000 5.7213
=====
ENTRENANDO CLASIFICADOR EM
=====
GRUPO 1
Media Intensidad : 171.74
Desviacion : 21.06
Cantidad Bordes : 623.85
Entropia : 5.61
GRUPO 2
Media Intensidad : 172.77
Desviacion : 19.85
Cantidad Bordes : 712.71
Entropia : 5.56
GRUPO 3
Media Intensidad : 174.50
Desviacion : 24.69
Cantidad Bordes : 453.03
Entropia : 5.56
GRUPO 4
Media Intensidad : 169.07
Desviacion : 14.31
Cantidad Bordes : 1158.00
Entropia : 5.77
GRUPO 5
Media Intensidad : 169.91
Desviacion : 15.25
Cantidad Bordes : 1265.00
Entropia : 5.86
GRUPO 6
Media Intensidad : 169.01
Desviacion : 14.08
Cantidad Bordes : 1108.00
Entropia : 5.74
GRUPO 7
Media Intensidad : 173.18
Desviacion : 8.43
Cantidad Bordes : 1343.00
Entropia : 5.01
GRUPO 8
Media Intensidad : 174.56
Desviacion : 7.91
Cantidad Bordes : 1513.00
Entropia : 4.91
GRUPO 9
Media Intensidad : 173.42
Desviacion : 8.07
Cantidad Bordes : 1392.00
Entropia : 4.90
=====
CLASIFICACION DE IMAGENES
=====
abejaMiel1.jpg ---> GRUPO 1
abejaMiel2.jpg ---> GRUPO 2
abejaMiel3.jpg ---> GRUPO 2
abejaMiel5.jpg ---> GRUPO 2
abejaPata1.jpg ---> GRUPO 3
abejaPata2.jpg ---> GRUPO 3
abejaPata3.jpg ---> GRUPO 3
abejaPata4.jpg ---> GRUPO 3
langosta1.jpg ---> GRUPO 4
langosta2.jpg ---> GRUPO 5
langosta3.jpg ---> GRUPO 6
libelula1.jpg ---> GRUPO 7
libelula2.jpg ---> GRUPO 8
libelula3.jpg ---> GRUPO 9
mariposa1.jpg ---> GRUPO 2
```

```

mariposa2.jpg ---> GRUPO 2
mariposa3.jpg ---> GRUPO 2
mosca1.jpg ---> GRUPO 2
mosca2.jpg ---> GRUPO 2
mosca3.jpg ---> GRUPO 2
=====
PRUEBA CON NUEVA IMAGEN
=====
Características imagen:
    162.5860    20.6951    709.0000    5.6080

Resultado:
La imagen pertenece al GRUPO 2

GRUPO 2
Media Intensidad : 172.77
Desviacion       : 19.85
Cantidad Bordes  : 712.71
Entropia         : 5.56
>> |

```

Como vemos en las capturas, el sistema funciono de forma correcta y clasifico a las muestras conforme a lo que “observo”.

- Primero leyó 20 imágenes y a cada una le extrajo 4 números (Media, Desviación, Bordes y Entropía).
- Con esos números, el algoritmo agrupo las imágenes por similitud y termino creando 9 Grupos.
- Asignó cada una de las 20 imágenes a uno de esos 9 grupos.
- Por último, proceso la imagen nueva y respecto a sus características le asigno un grupo, el cual fue el 2.

Es notable como el sistema logro identificar correctamente las abejaMiel y la abejaPata, para a cada una ponerlas en grupos separados junto con las otras imágenes de sus mismas partes. Además, que al ingresar la nueva imagen pudo clasificarla correctamente en el grupo 2.

OBSERVACIONES

Aunque el sistema funciona correctamente, podemos notar que hay variaciones en lo que el sistema detecta en imágenes de las mismas partes de insectos que deberían ir en un mismo grupo, como los son las alas de mariposas y libélulas, pero las pone cada una en uno distinto. También puede haber confusión de parte del sistema al clasificar las moscas, ya que las coloco en el mismo grupo de la abeja miel. Esto por los mínimos detalles diferentes que puede llegar a detectar entre las fotografías de una misma parte del insecto.

CONCLUSIONES

Al llevar a cabo el proyecto nos pudimos dar cuenta de todo lo que hay detrás de crear una IA que clasifique las imágenes, como lo son las matemáticas, promedios, fórmulas de distancia y variables como la confianza o la tasa de aprendizaje. Resulto bastante interesante ver el proceso de cómo es que los sistemas pueden extraer la información de las imágenes en distintos y complejos pasos, para después interpretarla y mediante límites de confianza agruparlas. También vimos cómo es que mediante la tasa de aprendizaje los sistemas se entrenan y aprenden de sus “errores”.

Sin duda, estos sistemas tienen un gran potencial y pueden ser de gran ayuda en campos como la medicina, informática, área industrial, etc. En donde se necesite analizar datos complejos para tomar una decisión.